

Claude Code

A Practical Guide for Developers

What is Claude Code?

- ◆ AI-powered command line tool for developers
- ◆ Lives directly in your terminal
- ◆ Understands your entire codebase
- ◆ Can read, write, and modify files
- ◆ Executes tests, builds, and git operations

Key Features

- ◆ File Operations: Read, write, edit any file
- ◆ Code Understanding: Ask questions about architecture
- ◆ Testing & Debugging: Run tests, fix errors automatically
- ◆ Git Integration: Manage commits, PRs, merge conflicts
- ◆ Project Analysis: Understand dependencies and structure
- ◆ Context Management: Maintains awareness across sessions

Common Use Cases

Daily Development

- Add features
- Fix bugs
- Write tests
- Refactor code
- Update docs

Code Analysis

- Understand legacy code
- Find security issues
- Review PRs
- Analyze dependencies

Project Management

- Create migration plans
- Handle merge conflicts
- Generate API docs
- Setup new features

Learning

- Explore new codebases
- Understand patterns
- Learn best practices
- Ask architecture questions

Installation

Native Installation:

Homebrew (macOS, Linux):

```
brew install --cask claude-code
```

macOS, Linux, WSL:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

Windows CMD:

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd  
&& del install.cmd
```

More info: <https://code.claude.com/docs/en/setup>

Initialisation

Requirements:

- Terminal access
- Account at <https://claude.ai/>

Authentication:

- Navigate to your project directory
- Run: `claude`
- Follow browser authentication

Essential Commands

Starting Sessions:

`claude`

→ Start interactive session

`claude "query"`

→ Start with initial prompt

`claude -c`

→ Continue last conversation

`claude —resume`

→ Choose from previous sessions to continue

`claude -p "query"`

→ Print mode (no interaction)

In-Session Commands:

`/clear`

→ Reset context window

`/rewind`

→ Go back to pervious checkpoint

`@filename`

→ Reference specific files

`/help`

→ Show all commands

CLAUDE.md - Your Project's Memory

What is it?

A special file that Claude reads on every session start

What to include:

- Project architecture and structure
- Coding standards and conventions
- Technology stack details
- File organization patterns
- Common workflows and processes

Tips:

- `/init` → Command to ask Claude Code to create CLAUDE.md file
- Keep it updated! Ask Claude to document decisions

Tips & Tricks

- ◆ Use /clear between tasks
- ◆ Do not be afraid to interrupt Claude
- ◆ Keep CLAUDE.md focused (not a dump)
- ◆ Ask Claude to create a plan in a .md file for a large tasks
- ◆ Be specific in prompts
 - ✓ «Refactor getUserData() for better error handling»
 - ✗ «Improve this function»
- ◆ Use --dangerously-skip-permissions for faster workflow (YOLO mode)

Advanced Features

Create custom / commands:

- /test → Run all tests and report
- /deploy → Deployment workflow

Specialised subagents:

- Code reviewer
- Test engineer
- Documentation writer

Use /agents to set up

Hooks

Execute scripts at key moments:

- Before/after tool use
- On notifications
- On session stop

MCP Integration

Connect to external tools:

- Brave Search
- GitHub
- Jira / Linear
- Custom services

Example:

```
claude mcp add brave-search
```

Permission Management

Configure in `.claude/settings.json`

Questions?